

**Herramientas de calidad de software aplicadas al caso Therac-25:
análisis, comparación y aplicación de SonarQube**

Breyner Alexander Pérez Álvarez

Corporación Universitaria Iberoamericana
Ingeniería de Software
Calidad de Software

Docente: Willian Ruiz

IBERO

**CORPORACIÓN UNIVERSITARIA
IBEROAMERICANA**

Actividad 4: Herramientas y tecnologías de calidad de software

03 octubre de 2026

Herramientas de calidad de software aplicadas al caso Therac-25

Introducción

La calidad del software no se alcanza al final del desarrollo: se construye y se verifica en cada etapa del ciclo de vida. Pressman y Maxim (2021) presentan el aseguramiento de la calidad como un conjunto de actividades planeadas y sistemáticas que buscan evidencia de que el producto cumple los requisitos y es confiable. Para llevarlas a la práctica, los equipos se apoyan en herramientas que automatizan pruebas, analizan el código fuente y gestionan los defectos.

Este informe continúa con el caso de estudio de la Actividad 2, el acelerador lineal médico Therac-25, cuyas fallas de software causaron sobredosis de radiación en pacientes entre 1985 y 1987 (Leveson & Turner, 1993). El propósito es investigar herramientas de calidad vigentes, compararlas con criterios definidos, seleccionar la más pertinente para el caso y describir cómo se aplicaría para detectar y prevenir los problemas que originaron la tragedia. El documento sigue el procedimiento de la actividad: selección de herramientas, análisis de características, comparación, ejecución práctica descriptiva y consolidación de conclusiones y recomendaciones.

Objetivos

Objetivo general. Seleccionar y aplicar de forma descriptiva una herramienta de calidad de software sobre el caso Therac-25, evaluando su impacto en la prevención de las fallas identificadas.

Objetivos específicos.

- Investigar y comparar herramientas de pruebas automatizadas, análisis estático y gestión de pruebas.
- Justificar la selección de una herramienta a partir de sus características, funcionalidades y ventajas.
- Describir su aplicación sobre un módulo simulado equivalente al del caso y proponer medidas preventivas.

1. Caso de estudio: el Therac-25

El Therac-25 fue un acelerador lineal de uso médico fabricado por Atomic Energy of Canada Limited (AECL) que ofrecía dos modos de tratamiento: haz de electrones de baja energía y rayos X de alta energía mediante un blanco de tungsteno. Entre junio de 1985 y enero de 1987 se documentaron seis accidentes en los que pacientes recibieron dosis muy superiores

a las prescritas, con consecuencias graves y, en varios casos, fatales (Leveson & Turner, 1993).

La investigación de Leveson y Turner (1993) identificó que el problema no fue una causa única, sino una combinación de fallas técnicas y de proceso. La Tabla 1 resume las principales y las asocia con atributos de calidad del modelo de McCall, que Pressman y Maxim (2021) recogen entre los factores de calidad del producto.

Tabla 1

Fallas principales del Therac-25 y atributos de calidad afectados

Falla	Descripción	Atributo afectado
F1. Condición de carrera	Si el operador editaba rápidamente los parámetros de tratamiento, la tarea de configuración podía usar datos desactualizados y dejar el equipo en un estado inconsistente.	Fiabilidad, corrección
F2. Desbordamiento de contador	Una variable de un solo byte se incrementaba en cada ciclo; al desbordarse y valer cero, se omitía una verificación de posición del colimador.	Corrección, fiabilidad
F3. Eliminación de interbloqueos por hardware	Se retiraron los enclavamientos físicos del Therac-20 y se confió la seguridad al software.	Integridad, seguridad
F4. Reutilización sin revisión	Se reutilizó software de modelos anteriores cuyos errores estaban enmascarados por los interbloqueos de hardware.	Reusabilidad, mantenibilidad
F5. Mensajes de error crípticos	El equipo mostraba códigos como “MALFUNCTION 54” sin explicación, y los operadores aprendieron a ignorarlos.	Usabilidad
F6. Pruebas y revisión insuficientes	El software no tuvo revisión independiente ni pruebas integrales del sistema completo; la respuesta a los incidentes reportados fue deficiente.	Facilidad de prueba, verificabilidad

Nota. Elaborado a partir de Leveson y Turner (1993) y Pressman y Maxim (2021).

2. Selección y análisis de herramientas

Se investigaron cinco herramientas representativas de tres categorías de la actividad: análisis estático, pruebas automatizadas y gestión de pruebas. La selección busca cubrir distintos momentos del proceso de calidad y no solo comparar productos similares.

2.1 Herramientas investigadas

Tabla 2

Características, funcionalidades y problemas que aborda cada herramienta

Herramienta	Categoría	Funcionalidades principales	Problemas que puede abordar
SonarQube (SonarSource)	Análisis estático	Analiza el código sin ejecutarlo y reporta bugs, vulnerabilidades, puntos críticos de seguridad y code smells; mide duplicación, complejidad y deuda técnica; aplica <i>quality gates</i> ; se integra con sistemas de CI.	Errores de lógica, uso inseguro de datos, código duplicado o confuso, ausencia de cobertura en código nuevo.
JUnit	Pruebas unitarias (Java)	Marco de pruebas automatizadas con aserciones, ciclo de vida de pruebas y ejecución desde Maven, Gradle o el IDE.	Regresiones y errores en la lógica de unidades pequeñas de código.
Selenium	Pruebas funcionales de interfaz web	Automatiza navegadores para simular la interacción del usuario con aplicaciones web.	Fallas de flujo en interfaces web; no analiza código ni sistemas embebidos.
TestRail	Gestión de pruebas	Organiza casos, planes y ejecuciones de prueba; genera informes de trazabilidad y avance.	Falta de trazabilidad entre requisitos, pruebas y defectos.
SoapUI (SmartBear)	Pruebas de servicios web/API	Crea y ejecuta pruebas funcionales, de carga y de seguridad sobre servicios SOAP y REST.	Defectos en la comunicación entre servicios y contratos de API.

Nota. Elaborado a partir de la documentación oficial de cada herramienta (SonarSource, s. f.; JUnit, s. f.; Selenium, s. f.; SmartBear Software, 2024).

2.2 Comparación

Para comparar se emplearon los criterios de la actividad (facilidad de uso, costo, soporte de plataformas, integración y comunidad) y se añadió un sexto: la pertinencia respecto al caso Therac-25. Cada criterio se valoró de 1 (bajo) a 5 (alto). La valoración es un juicio razonado del autor a partir de la documentación consultada, no una medición objetiva, y su finalidad es apoyar la decisión de forma transparente.

Tabla 3

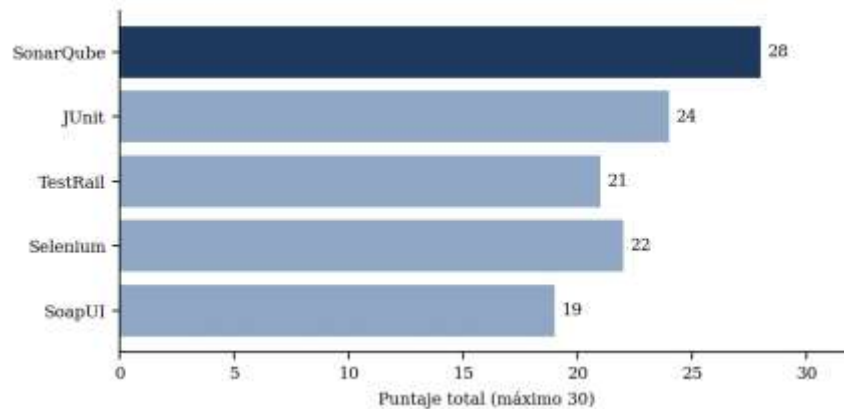
Matriz de comparación (1 = bajo, 5 = alto)

Criterio	SonarQube	JUnit	TestRail	Selenium	SoapUI
Facilidad de uso	4	4	4	3	4
Costo (5 = más accesible)	4	5	2	5	4
Soporte de plataformas y lenguajes	5	3	5	4	4
Integración con el proceso de desarrollo	5	4	4	4	3
Comunidad y soporte	5	5	3	5	3
Pertinencia para el caso Therac-25	5	3	3	1	1
Total (máx. 30)	28	24	21	22	19

Nota. Valoración propia. En costo, SonarQube cuenta con una edición comunitaria gratuita y ediciones comerciales; TestRail es un producto de pago por suscripción; JUnit y Selenium son de código abierto.

Figura 1

Puntaje total por herramienta según la matriz de comparación



Nota. Elaboración propia.

2.3 Ventajas y desventajas según el contexto

- **SonarQube.** Ventaja: detecta defectos tempranamente, de forma automática y repetible, y produce métricas objetivas. Desventaja: genera falsos positivos, requiere configurar reglas y no sustituye las pruebas dinámicas.

- **JUnit.** Ventaja: sencillo y muy extendido para verificar la lógica. Desventaja: solo es tan bueno como los casos escritos y no revela por sí solo problemas de concurrencia.
- **TestRail.** Ventaja: da trazabilidad y control del proceso de pruebas. Desventaja: costo y esfuerzo de mantenimiento, y no analiza el código.
- **Selenium y SoapUI.** Ventaja: excelentes en su nicho (interfaz web y APIs). Desventaja: el caso Therac-25 es un sistema embebido de control, por lo que su aporte directo es limitado.

3. Herramienta seleccionada: SonarQube

Se selecciona SonarQube por obtener el mayor puntaje en la matriz y, sobre todo, por su relación directa con la naturaleza de las fallas del caso. Casi todas las causas técnicas del Therac-25 (acceso concurrente a datos compartidos, desbordamiento de contador, código reutilizado sin revisar) son defectos que residen en el código fuente y que el análisis estático está diseñado para señalar antes de que el software llegue a operar.

3.1 Características y funcionalidades relevantes

- **Reglas de calidad y seguridad:** clasifica los hallazgos en bugs, vulnerabilidades, puntos críticos de seguridad y code smells, con indicación de severidad y de cómo corregirlos.
- **Métricas de mantenibilidad:** complejidad, duplicación de código y estimación de deuda técnica, útiles para evaluar el riesgo de reutilizar código heredado.
- **Quality gate:** conjunto de condiciones que el código nuevo debe cumplir para considerarse aceptable; si no las cumple, el cambio no avanza.
- **Integración continua:** se ejecuta automáticamente en cada commit o pull request desde sistemas de CI, y muestra el resultado a todo el equipo.
- **Cobertura de pruebas:** importa los reportes de herramientas como JUnit y evidencia qué código no está siendo probado.

3.2 Limitaciones que deben reconocerse

Un análisis crítico exige señalar los límites de la herramienta. Primero, el software original del Therac-25 estaba escrito en lenguaje ensamblador para un procesador PDP-11 (Leveson & Turner, 1993), un entorno que SonarQube no analiza. Por esa razón, la aplicación de este informe se hace sobre un módulo simulado, escrito en un lenguaje actual, que reproduce el tipo de defecto del caso; la actividad admite expresamente un proyecto real o simulado. Segundo, la detección estática de condiciones de carrera es parcial: la herramienta puede

señalar patrones de acceso no sincronizado, pero no demuestra la ausencia de errores de concurrencia. Tercero, los resultados requieren interpretación humana por la posibilidad de falsos positivos. Por estas razones SonarQube se propone como una capa dentro de una estrategia de calidad más amplia y no como solución única.

4. Aplicación práctica sobre el caso (parte descriptiva)

Se describe cómo se usaría SonarQube en un proyecto simulado que representa el software de control de un equipo de radioterapia. Los hallazgos presentados son ilustrativos: describen el tipo de alerta que la herramienta emite ante cada patrón de defecto, y no corresponden a una ejecución real ni reproducen identificadores exactos de reglas.

4.1 Módulo simulado

El siguiente fragmento en lenguaje C condensa tres defectos inspirados en el caso: una variable compartida entre el manejador del teclado y la tarea de configuración sin ningún mecanismo de exclusión, un contador de un byte que al desbordarse omite una verificación de seguridad, y una condición que descarta silenciosamente un error.

Figura 2

Módulo simulado con defectos inspirados en el Therac-25

```
/* Modulo simulado - ilustrativo */ volatile int modo_tratamiento; /* escrito por
teclado, leído por config */ unsigned char contador_pasadas = 0; /* 1 byte: se
desborda al llegar a 256 */

void manejador_teclado(int nuevo_modo) { modo_tratamiento =
nuevo_modo; /* (A) sin sincronizacion */ }

void tarea_configuracion(void) { contador_pasadas++; /* (B) vuelve a 0
tras 255 */ if (contador_pasadas != 0) { /* (C) con 0 se omite la verificacion */
verificar_posicion_colimador(); } configurar_haz(modo_tratamiento); /* (A)
lectura de dato compartido */ }

int aplicar_dosis(void) { int estado = leer_estado_equipo(); if (estado ==
ERROR_54) { return 0; } /* (D) error ignorado sin registro */ return emitir_haz();
}
```

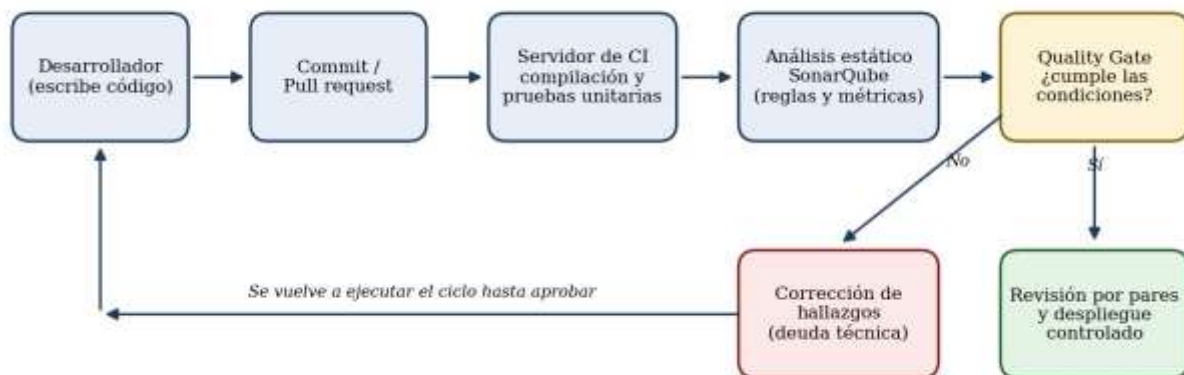
Nota. Código de elaboración propia con fines ilustrativos; no es el código original del Therac-25.

4.2 Proceso de calidad con la herramienta

El proceso se integra al flujo de desarrollo como muestra la Figura 3: cada cambio de código pasa por compilación y pruebas unitarias, luego por el análisis estático y finalmente por el quality gate. Si el gate falla, el cambio vuelve al desarrollador y no se despliega hasta que los hallazgos se corrijan.

Figura 3

Flujo de SonarQube dentro del proceso de desarrollo



Nota. Elaboración propia.

4.3 Hallazgos esperados y su relación con las fallas del caso

Tabla 4

Relación entre las fallas del Therac-25, los hallazgos esperados y las medidas de calidad

Falla del caso	Hallazgo esperado en el análisis (ilustrativo)	Medida correctiva y complementaria
F1. Condición de carrera (marca A)	Alerta por acceso a una variable compartida sin sincronización; posible bug de concurrencia en la lectura y escritura.	Proteger el dato con exclusión mutua o pasar mensajes entre tareas; agregar pruebas de concurrencia y de estrés con entradas rápidas.
F2. Desbordamiento (marcas B y C)	Alerta por aritmética que puede desbordar el tipo y por una condición que depende del valor envuelto del contador.	Usar un indicador booleano explícito en lugar de un contador; pruebas de valores límite; revisión de código.

F3. Falta de interbloqueos	No es detectable por análisis estático, porque es una decisión de diseño.	Análisis de peligros y riesgos en el diseño; mantener enclavamientos por hardware independientes del software.
F4. Reutilización sin revisión	Métricas de complejidad, duplicación y deuda técnica elevadas en el código heredado.	Revisar y probar el código reutilizado en su nuevo contexto antes de integrarlo.
F5. Errores crípticos y manejo (marca D)	Alerta por manejo de excepción o estado de error ignorado sin registro.	Registrar los errores y mostrar mensajes comprensibles para el operador; pruebas de usabilidad.
F6. Pruebas insuficientes	Cobertura baja en código nuevo; el quality gate falla.	Exigir cobertura mínima, pruebas de integración y revisión independiente; gestionar casos y defectos con trazabilidad.

Nota. Los hallazgos describen el tipo de alerta esperado y no una salida real de la herramienta. Elaboración propia a partir de Leveson y Turner (1993).

5. Evaluación de impacto y medidas anticipadas

5.1 Impacto en la resolución de problemas

De las seis fallas de la Tabla 4, SonarQube aporta una señal directa y temprana en cuatro de ellas (F1, F2, F5 y F6) y una señal indirecta en F4, mediante las métricas de deuda técnica. En F3 no aporta, ya que se trata de una decisión de arquitectura. El impacto principal es de proceso: el quality gate convierte la calidad del código en una condición verificable antes de cada integración, de modo que la omisión de pruebas o la reutilización sin revisión, que en el caso pasaron inadvertidas, quedarían visibles para el equipo y para los responsables.

Esta evaluación coincide con lo que señalan Leveson y Turner (1993): ninguna herramienta aislada habría evitado los accidentes, pues la causa de fondo fue una cultura de seguridad deficiente que dependía de una sola capa de protección. Por eso el valor de la herramienta está en integrarse a un sistema de defensas en profundidad.

5.2 Medidas anticipadas

Con base en el análisis, se proponen las siguientes medidas para un proyecto de software crítico como el del caso:

- **Quality gate estricto.** Exigir cero bugs y cero vulnerabilidades nuevas, calificaciones máximas de fiabilidad y seguridad, cobertura de pruebas sobre el código nuevo de al

menos 80 % y duplicación baja. La configuración por defecto de SonarQube (Sonar way) usa umbrales de ese orden (SonarSource, s. f.).

- **Integración continua obligatoria.** Ningún cambio se integra sin pasar compilación, pruebas y análisis estático.
- **Pruebas dinámicas complementarias.** Pruebas unitarias con JUnit o equivalentes, pruebas de concurrencia, de valores límite y de integración del sistema completo.
- **Revisión por pares independiente.** Revisión humana del código crítico y del código reutilizado, como complemento de la herramienta.
- **Análisis de riesgos y seguridad por diseño.** Identificación de peligros desde el diseño y mantenimiento de interbloqueos por hardware, siguiendo marcos como IEC 62304 para software de dispositivos médicos.
- **Gestión de pruebas y defectos.** Registro con trazabilidad entre requisitos, pruebas e incidentes, con una respuesta formal a cada falla reportada por los usuarios.
- **Comunicación clara al operador.** Mensajes de error descriptivos y procedimientos ante alarmas.

5.3 Riesgos de la propuesta

Los principales riesgos son confiar en exceso en el semáforo verde del quality gate, ignorar hallazgos por considerarlos falsos positivos sin analizarlos, y configurar umbrales tan laxos que pierdan utilidad. Se mitigan con revisión periódica de las reglas, con registro de las excepciones aprobadas y con la combinación de la herramienta con pruebas dinámicas y revisión humana.

Conclusiones

- La investigación y comparación de cinco herramientas muestra que no existe una herramienta universalmente mejor: la elección depende del tipo de defecto que se quiere prevenir. Para el caso Therac-25, cuyas fallas técnicas residían en el código y en el proceso, SonarQube fue la opción más pertinente por su análisis estático, sus métricas de deuda técnica y su quality gate.
- La aplicación descriptiva sobre un módulo simulado mostró que la herramienta habría podido señalar de forma temprana el acceso no sincronizado a datos compartidos, el desbordamiento del contador, el manejo deficiente de errores y la falta de cobertura de pruebas. También evidenció sus límites: no detecta decisiones de diseño como la eliminación de interbloqueos, no analiza lenguaje ensamblador y puede generar falsos positivos.

- La recomendación principal es usar SonarQube como una capa dentro de una estrategia de calidad integral, que combine integración continua, pruebas dinámicas, revisión independiente, análisis de riesgos y gestión de defectos. El Therac-25 recuerda que, en sistemas críticos, la calidad del software es una responsabilidad ética y que una sola barrera de protección nunca es suficiente.

Referencias

- Aizprua, S., Ortega, A., & Von Chong, L. (2019). Calidad del software, una perspectiva continua. *Revista Científica Universitaria*, 8(2), 1–18.
- JUnit. (s. f.). *JUnit 5 user guide*. <https://junit.org/junit5/docs/current/user-guide/>
- Leveson, N. G., & Turner, C. S. (1993). An investigation of the Therac-25 accidents.
 - *Computer*, 26(7), 18–41. <https://doi.org/10.1109/MC.1993.274940>
- Piattini-Velthuis, M., & García-Rubio, F. (2015). *Calidad de sistemas de información*.
 - RA-MA Editorial.
- Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software*. McGraw-Hill Interamericana.
- Selenium. (s. f.). *Selenium documentation*. <https://www.selenium.dev/documentation/>
- SmartBear Software. (2024). *SoapUI* [Sitio web]. <https://www.soapui.org>
- SonarSource. (s. f.). *SonarQube documentation*. <https://docs.sonarsource.com/sonarqube/>